Introduction to Parallel Computing and Architectures

Acknowledgements

Introduction to Parallel Computing", Addison Wesley, 2003
Ananth Grama, Anshul Gupta, George Karypis, and Vipin Kumar
Blaise Barney, Lawrence Livermore National Laboratory, Introduction to Parallel Computing

Outline

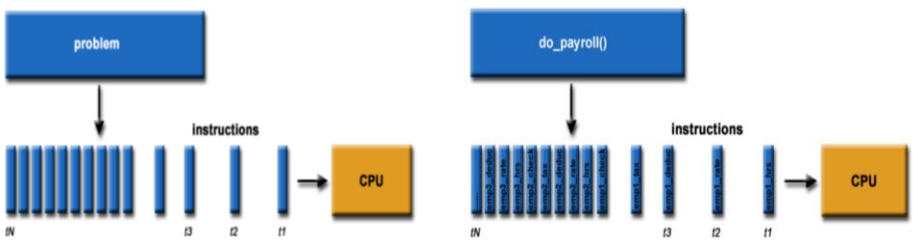
- Introduction
  - What is Parallel Computing
  - Why use Parallel Computing
  - Concepts and Terminology
- Parallel Computer Memory Architectures
- Parallel Programming Models
- Designing Parallel Programs

Outline

- **Introduction**
  - What is Parallel Computing
  - Why use Parallel Computing
  - Concepts and Terminology
- Parallel Computer Memory Architectures
- Parallel Programming Models
- Designing Parallel Programs

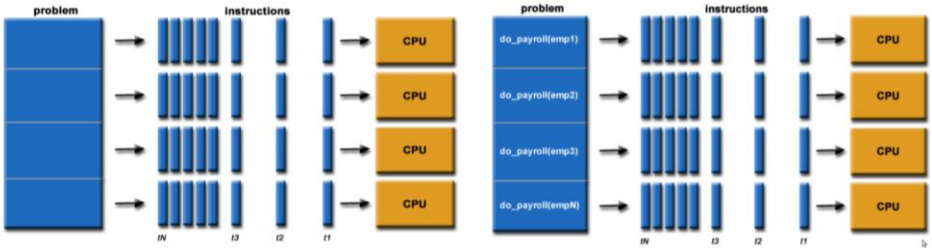
What is Parallel Computing?

- Traditionally, software has been written for serial computation:
  - To be run on a single computer having a single CPU.
  - A problem is broken into a discrete series of instructions.
  - Instructions are executed one after another.
  - Only one instruction may execute at any moment in time.



What is Parallel Computing?

- Parallel computing is the simultaneous use of multiple compute resources to solve a computational problem:
  - To be run using multiple CPUs.
  - A problem is broken into discrete parts that can be solved concurrently.
  - Each part is further broken down to a series of instructions.
  - Instructions from each part execute simultaneously on different CPUs.



What is Parallel Computing?

- The compute resources might be:
  - A single computer with multiple processors;
  - An arbitrary number of computers connected by a network;
  - A combination of both.
- The computational problem should be able to:
  - Be broken apart into discrete pieces of work that can be solved simultaneously;
  - Execute multiple program instructions at any moment in time;
  - Be solved in less time with multiple compute resources than with a single compute resource.
- Parallel computing has been used to model difficult problems:
  - Atmosphere, Earth, Environment, Physics, Bioscience, Biotechnology, Genetics, Chemistry, Computer Science, Mathematics

Why Use Parallel Computing

- Save time and/or money: In theory, throwing more resources at a task will shorten its time to completion, with potential cost savings.
- Solve larger problems: Many problems are so large and/or complex that it is impractical or impossible to solve them on a single computer, especially given limited computer memory (i.e., https://en.wikipedia.org/wiki/Grand_Challenges).
- Provide concurrency: A single compute resource can only do one thing at a time. Multiple computing resources can be doing many things simultaneously.
- Use of non-local resources: Using compute resources on a wide area network, or even the Internet when local compute resources are scarce.

Why Use Parallel Computing

- During the past 20+ years, the trends indicated by ever faster networks and multi-processor computer architectures (the desktop level) clearly show that parallelism is the future of computing.
- In this same time period, there has been a greater than 1000x increase in supercomputer performance.
- The race is already on for Exscale Computing. The global race to build practical quantum computers is accelerating.

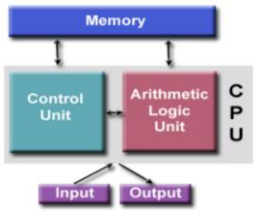
Feature	Exascale Computing	Quantum Computing
Core Technology	Advanced silicon transistors (Bits: 0 or 1)	Quantum mechanics (Qubits: Superposition and Entanglement)
Processing Style	Massive parallel processing	Exploring many possible solutions using quantum effects
Current Status	Fully operational. Used for climate modeling, defense, and AI.	Experimental / Hybrid. Accessible through the cloud; practical applications are still developing.
Best Used For	Weather modeling, deep learning, and large-scale engineering	Molecular chemistry, cryptography, complex logistics, and material design
Main Limit	High power consumption, cooling requirements, and physical hardware limits	Sensitive hardware requiring very controlled conditions and extremely low temperatures

Computer performance

Name	FLOPS
yottaFLOPS	10^{24}
zettaFLOPS	10^{21}
exaFLOPS	10^{18}
petaFLOPS	10^{15}
teraFLOPS	10^{12}
gigaFLOPS	10^{9}
megaFLOPS	10^{6}
kiloFLOPS	10^{3}

Concepts and Terminology

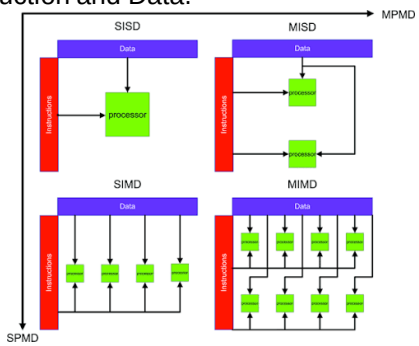
- Virtually all computers have followed von Neumann Architecture.
  - Comprised of four main components: Memory, Control Unit, Arithmetic Logic Unit, Input/Output
  - Read/write, random access memory is used to store both program instructions and data
    - » Program instructions are coded data which tell the computer to do something
    - » Data is simply information to be used by the program
  - Control unit fetches instructions/data from memory, decodes the instructions and then sequentially coordinates operations to accomplish the programmed task.
  - Aritmetic Unit performs basic arithmetic operations
  - Input/Output is the interface to the human operator



Concepts and Terminology

- Parallel computers still follow this basic design, just multiplied in units. The basic, fundamental architecture remains the same.
- Flynn's taxonomy distinguishes multi-processor computer architectures according to how they can be classified along the two independent dimensions of Instruction and Data.

	Single Data (SD)	Multiple Data (MD)
Single Instruction (SI)	SISD: Single Instruction, Single Data	SIMD: Single Instruction, Multiple Data
Multiple Instruction (MI)	MISD: Multiple Instruction, Single Data	MIMD: Multiple Instruction, Multiple Data

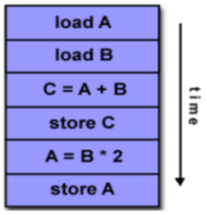


Concepts and Terminology

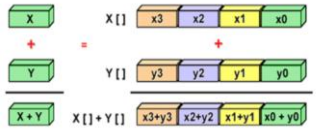
- Single Instruction, Single Data
  - A serial (non-parallel) computer
  - Single Instruction: Only one instruction stream is being acted on by the CPU during any one clock cycle
  - Single Data: Only one data stream is being used as input during any one clock cycle
  - Deterministic execution
  - This is the oldest and even today, the most common type of computer
  - Examples: older generation mainframes, minicomputers and workstations; most modern day PCs.



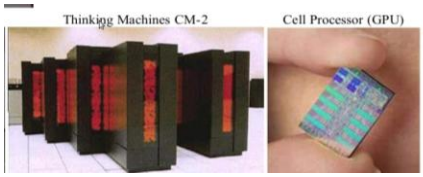


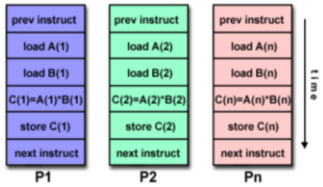


Concepts and Terminology



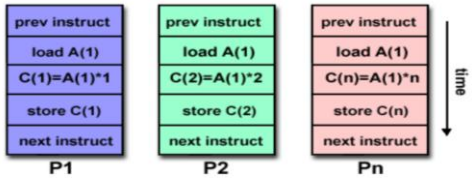
- Single Instruction, Multiple Data
  - A type of parallel computer
  - Processing units execute the same instruction at any given clock cycle
  - Each processing unit can operate on a different data element
  - Best suited for specialized problems characterized by a high degree of regularity, such as graphics/image processing.
  - Synchronous (lockstep) and deterministic execution
  - Two varieties: Processor Arrays (e.g., MasPar MP-1 & MP-2) and Vector Pipelines (e.g., IBM 9000, Cray X-MP).
  - Most modern computers, particularly those with graphics processor units (GPUs) employ SIMD instructions and execution units.





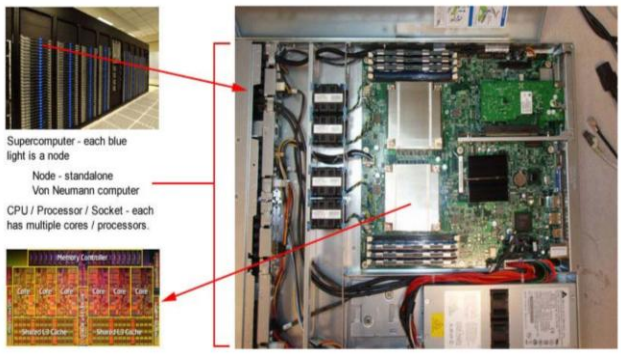
Concepts and Terminology

- Multiple Instruction, Single Data (MISD):
  - A type of parallel computer
  - Each processing unit operates on the data independently via separate instruction streams.
  - A single data stream is fed into multiple processing units.
  - Few actual examples of this class of parallel computer have ever existed. One is the experimental Carnegie-Mellon C.mmp computer (1971).
  - Some conceivable uses might be:
    - multiple frequency filters operating on a single signal stream
    - multiple cryptography algorithms attempting to crack a single coded message.



Concepts and Terminology

- **Supercomputing** / High Performance Computing (HPC) is an area that use the world's fastest and largest computers to solve large problems.
- **Node** is a standalone "computer in a box". Usually comprised of multiple CPUs/processors/cores. The result is a node with multiple CPUs, each containing multiple cores. Nodes are networked together to comprise a supercomputer.



Concepts and Terminology

- Multiple Instruction, Multiple Data (MIMD):
  - A type of parallel computer
  - Every processor may be executing a different instruction stream
  - Every processor may be working with a different data stream
  - Currently, the most common type of parallel computer - most modern supercomputers fall into this category.
  - Most current supercomputers, networked parallel computer clusters and "grids", multi-processor SMP computers, multi-core PCs.
  - Many MIMD architectures also include SIMD execution sub-components



Concepts and Terminology

- **Task**, a logically discrete section of computational work. A task is typically a program or program-like set of instructions that is executed by a processor.
- **Parallel program** consists of multiple tasks running on multiple processors.
- **Pipelining** is the process of breaking a task into steps performed by different processor units, with inputs streaming through, much like an assembly line; a type of parallel computing.
- **Shared Memory**, describes a model where parallel tasks all have the same "picture" of memory and can directly address and access the same logical memory locations regardless of where the physical memory actually exists.
- **Distributed Memory**, tasks can only logically "see“ local machine memory and must use communications to access memory on other machines where other tasks are executing.

Concepts and Terminology

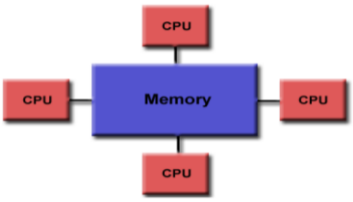
- **Communications**, parallel tasks typically need to exchange data. There are several ways this can be accomplished, such as through a shared memory bus or over a network, however the actual event of data exchange is commonly referred to as communications regardless of the method employed.
- **Synchronization**, the coordination of parallel tasks in real time, very often associated with communications. Often implemented by establishing a synchronization point within an application where a task may not proceed further until another task(s) reaches the same or logically equivalent point.
- **Parallel Overhead**, the amount of time required to coordinate parallel tasks, as opposed to doing useful work. Parallel overhead can include factors such as: Task start-up time, Synchronizations, Data communications, Software overhead imposed by parallel compilers, libraries, tools, operating system, etc, and Task termination time.

Outline

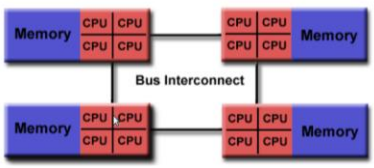
- Introduction
  - What is Parallel Computing
  - Why use Parallel Computing
  - Concepts and Terminology
- **Parallel Computer Memory Architectures**
- Parallel Programming Models
- Designing Parallel Programs

Parallel Computer Memory Architectures

- Shared memory parallel computers vary widely, but generally have in common the ability for all processors to access all memory as global address space.
- Multiple processors can operate independently but share the same memory resources.
- Changes in a memory location effected by one processor are visible to all other processors.
- Shared memory machines can be divided into two main classes based upon memory access times: UMA and NUMA.



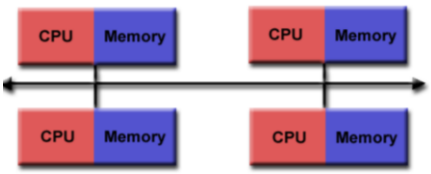
Shared Memory (UMA)



Shared Memory (NUMA)

Parallel Computer Memory Architectures

- **Distributed memory systems** vary widely but share a common characteristic. Distributed memory systems require a communication network to connect inter-processor memory.
- Memory addresses in a processor do not map to another processor, there is no concept of global address space across all processors.
- Because each processor has its own local memory, it operates independently. Changes it makes to its local memory have no effect on the memory of other processors.
- When a processor needs access to data in another processor, it is usually the task of the programmer to explicitly define how and when data is communicated. Synchronization between tasks is likewise the programmer's responsibility.



Outline

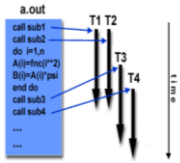
- Introduction
  - What is Parallel Computing
  - Why use Parallel Computing
  - Concepts and Terminology
- Parallel Computer Memory Architectures
- **Parallel Programming Models**
- Designing Parallel Programs

Parallel Programming Models

- There are several parallel programming models in common use:
  - Shared Memory
  - Distributed Memory / Message Passing
  - Hybrid
  - Single Program Multiple Data (SPMD)
  - Multiple Program Multiple Data (MPMD)
- Parallel programming models exist as an abstraction above hardware and memory architectures.
  - SHARED memory model on a DISTRIBUTED memory machine.
  - DISTRIBUTED memory model on a SHARED memory machine.
  - There is no "best" model, although there certainly are better implementations of some models over others.

Parallel Programming Models
Shared Memory Model

- A type of shared memory programming in which a single process can have multiple, concurrent execution paths.
- A single program includes a number of subroutines:
  - The main program *a.out* is scheduled to run by the native operating system. *a.out* loads and acquires all of the necessary system and user resources to run.
  - *a.out* performs some serial work, and then creates a number of tasks (threads) that can be scheduled and run concurrently.
  - Each thread has local data, but also, shares the entire resources of *a.out* . Each thread benefits from a global memory view because it shares the memory space of *a.out* .
  - Any thread can execute any subroutine at the same time as other threads.

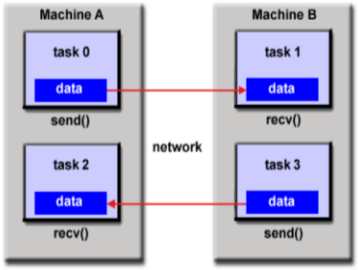


Parallel Programming Models
Shared Memory Model

- From a programming perspective, threads implementations commonly comprise:
  - A library of subroutines that are called from within parallel source code
  - A set of compiler directives imbedded in either serial or parallel source code
  - Unrelated standardization efforts have resulted in two very different implementations of threads: POSIX Threads and OpenMP.
    - **POSIX Threads** is library based; requires parallel coding
      - Specified by the IEEE POSIX 1003.1c standard (1995).
      - C Language only and commonly referred to as Pthreads.
      - Most hardware vendors now offer Pthreads in addition to their proprietary threads implementations.
      - Very explicit parallelism; requires significant programmer attention to detail.
      - POSIX Threads tutorial: computing.llnl.gov/tutorials/pthreads
    - **OpenMP** is compiler directive based; can use serial code
      - Jointly defined and endorsed by a group of major computer hardware and software vendors. Portable / multi-platform, including Unix and Windows NT platforms
      - Available in C/C++ and Fortran implementations
      - Can be very easy and simple to use - provides for "incremental parallelism"
      - OpenMP tutorial: computing.llnl.gov/tutorials/openMP

Parallel Programming Models
Distributed Memory / Message Passing Model

- A set of tasks that use their own local memory during computation. Multiple tasks can reside on the same physical machine and/or across an arbitrary number of machines.
- Tasks exchange data through communications by sending and receiving messages.
- Data transfer usually requires cooperative operations to be performed by each process. For example, a send operation must have a matching receive operation.



Parallel Programming Models
Distributed Memory / Message Passing Model

- From a programming perspective, message passing implementations usually comprise a library of subroutines. Calls to these subroutines are imbedded in source code. The programmer is responsible for determining all parallelism.

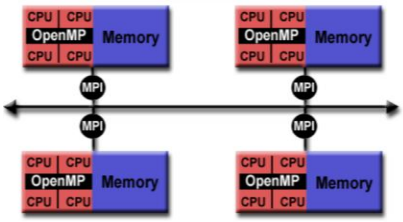
- MPI is now the "de facto" industry standard for message passing, replacing virtually all other message passing implementations used for production work.

- MPI implementations exist for virtually all popular parallel computing platforms.

- MPI tutorial: computing.llnl.gov/tutorials/mpi

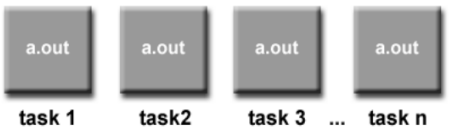
Parallel Programming Models
Hybrid Model

- A hybrid model combines more than one of the previously described programming models.
- Currently, a common example of a hybrid model is the combination of the MPI with the threads model (OpenMP).
  - Threads perform computationally intensive kernels using local data
  - Communications between processes on different nodes occurs over the network using MPI
- Another similar and increasingly popular example of a hybrid model is using MPI with GPU (Graphics Processing Unit) programming.



Parallel Programming Models
SPMD and MPMD

- SPMD is actually a "high level" programming model that can be built upon any combination of the previously mentioned parallel programming models.
  - All tasks execute their copy of the same program simultaneously. This program can be threads, message passing or hybrid.
  - All tasks may use different data.
- SPMD programs usually have the necessary logic programmed into them to allow different tasks to branch or conditionally execute only those parts of the program they are designed to execute.
- The SPMD model, using message passing or hybrid programming, is probably the most commonly used parallel programming model for multi-node clusters.



Parallel Programming Models
SPMD and MPMD

- MPMD is actually a "high level" programming model that can be built upon any combination of the previously mentioned parallel programming models.
  - Tasks may execute different programs simultaneously. The programs can be threads, message passing, data parallel or hybrid.
  - All tasks may use different data.
- MPMD applications are not as common as SPMD applications, but may be better suited or certain types of problems, particularly those that lend themselves better to functional decomposition than domain decomposition.



Outline

- Introduction
  - What is Parallel Computing
  - Why use Parallel Computing
  - Concepts and Terminology
- Parallel Computer Memory Architectures
- Parallel Programming Models
- **Designing Parallel Programs**

Designing Parallel Programs

- Designing and developing parallel programs has characteristically been a very manual process. The programmer is typically responsible for both identifying and actually implementing parallelism.

- Very often, manually developing parallel codes is a time consuming, complex, error-prone and iterative process.

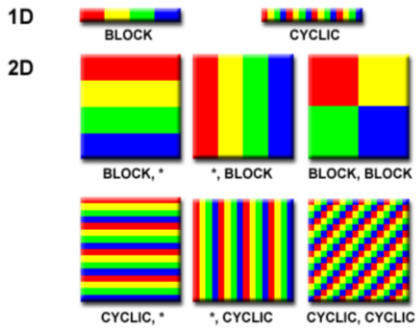
- The most common type of tools used to automatically parallelize a serial program is a parallelizing compiler or pre-processor:
  - Fully Automatic: the compiler analyzes the source code and identifies opportunities (e.g., loops) for parallelism.
  - Programmer Directed: the programmer explicitly tells the compiler how to parallelize the code.

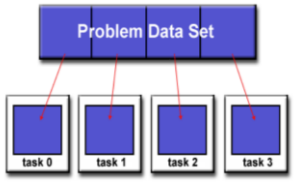
Designing Parallel Programs

- **Understand the problem and the program**
  - Determine whether or not the problem is one that can actually be parallelized, e.g., matrix-vector multiplication, calculation of the Fibonacci series (F(n) = F(n-1) + F(n-2)), …
  - Identify the program's hotspots: profilers and performance analysis tools can help here.
  - Identify bottlenecks in the program: I/O is usually something that slows a program down.
  - Identify inhibitors to parallelism: one common class of inhibitor is data dependence, e.g., Fibonacci.
  - Investigate other algorithms if possible.: this may be the single most important consideration when designing a parallel application.

Designing Parallel Programs

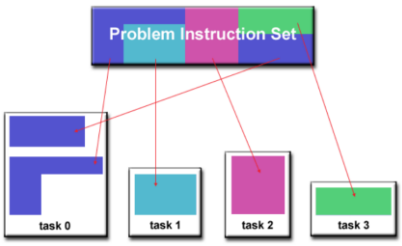
- **Partitioning/decomposition**
  - Break the problem into discrete "chunks" of work that can be distributed to multiple tasks.
    - ***Domain Decomposition***: the data associated with a problem is decomposed. Each parallel task then works on a portion of the data.

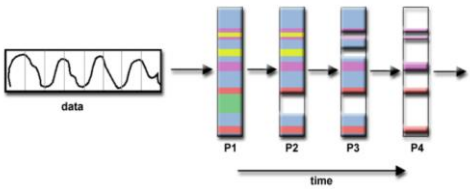




Designing Parallel Programs

- ***Functional Decomposition***: the focus is on the computation that is to be performed rather than on the data manipulated by the computation. The problem is decomposed according to the work that must be done. Each task then performs a portion of the overall work.





An audio signal data set is passed through four distinct computational filters. Each filter is a separate process.

Designing Parallel Programs

- **Communications:** The need for communications between tasks depends upon the problem being solved.
  - Some types of problems can be decomposed and executed in parallel with virtually no need for tasks to share data, e.g., an image processing operation where every pixel in a black and white image needs to have its color reversed.
  - Most parallel applications do require tasks to share data with each other, e.g., a 3-D heat diffusion problem requires a task to know the temperatures calculated by the tasks that have neighboring data.
  - Factors to Consider:
    - Inter-task communication virtually always implies overhead (e.g., resources used to package and transmit data, synchronization, available network bandwidth.
    - Visibility of communications: With the Message Passing Model, communications are explicit, with the Data Parallel Model, communications often occur transparently

Designing Parallel Programs

  - Factors to Consider:
    - Latency is the time it takes to send a minimal (0 byte) message from point A to point B. Sending many small messages can cause latency to dominate communication overheads.
    - Synchronous vs. asynchronous communications:
      - Synchronous communications are often referred to as blocking communications since other work must wait until the communications have completed.
      - Asynchronous communications are often referred to as non-blocking communications since other work can be done while the communications are taking place.
    - Scope of communications: knowing which tasks must communicate with each other is critical during the design stage of a parallel code.
      - Point-to-point - involves two tasks with one task acting as the sender/producer of data, and the other acting as the receiver/consumer.
      - Collective - involves data sharing between more than two tasks, which are often specified as being members in a common group, or collective.



Designing Parallel Programs

- **Synchronization**
  - Barrier
    - Usually implies that all tasks are involved
    - Each task performs its work until it reaches the barrier. It then stops, or "blocks".
    - When the last task reaches the barrier, all tasks are synchronized.
    - What happens from here varies. Often, a serial section of work must be done. In other cases, the tasks are automatically released to continue their work.
  - Lock / semaphore
    - Typically used to serialize (protect) access to global data or a section of code. Only one task at a time may use (own) the lock / semaphore / flag.
    - The first task to acquire the lock "sets" it. This task can then safely (serially) access the protected data or code.
    - Other tasks can attempt to acquire the lock but must wait until the task that owns the lock releases it.

Designing Parallel Programs

- **Data Dependencies**
  - A data dependence results from multiple use of the same location(s) in storage by different tasks.
    - Distributed memory architectures - communicate required data at synchronization points.
    - Shared memory architectures -synchronize read/write operations between tasks.
  - Although all data dependencies are important to identify when designing parallel programs, loop carried dependencies are particularly important since loops are possibly the most common target of parallelization efforts.
    - Distributed memory architectures - communicate required data at synchronization points.
    - Shared memory architectures -synchronize read/write operations between tasks.

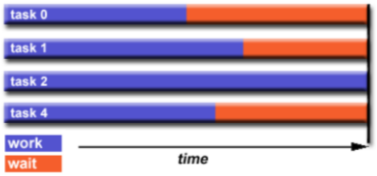
```
DO 500 J = MYSTART,MYEND
   A(J) = A(J-1) * 2.0
500 CONTINUE
```

```
task 1          task 2
------          ------

X = 2           X = 4
  .               .
  .               .
Y = X**2        Y = X**3
```

Designing Parallel Programs

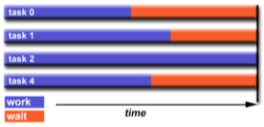
- **Load Balancing**
  - Load balancing refers to the practice of distributing work among tasks so that all tasks are kept busy all of the time. It can be considered a minimization of task idle time.
  - Load balancing is important to parallel programs for performance reasons. For example, if all tasks are subject to a barrier synchronization point, the slowest task will determine the overall performance.
    - Equally partition the work each task receives (e.g., array/matrix operations)
    - Use dynamic work assignment (e.g., Sparse arrays). It may become necessary to design an algorithm which detects and handles load imbalances as they occur dynamically within the code.



- How to Handle Data Dependencies:
  - Distributed memory architectures - communicate required data at synchronization points.
  - Shared memory architectures -synchronize read/write operations between tasks.

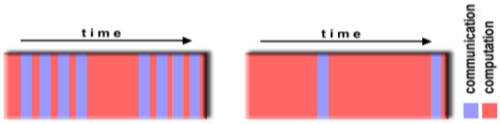
Designing Parallel Programs

- **How to Achieve Load Balance:**
  - Equally partition the work each task receives.
    - For array/matrix operations where each task performs similar work, evenly distribute the data set among the tasks.
    - For loop iterations where the work done in each iteration is similar, evenly distribute the iterations across the tasks.
    - If a heterogeneous mix of machines with varying performance characteristics are being used, be sure to use some type of performance analysis tool to detect any load imbalances. Adjust work accordingly.
  - Use dynamic work assignment:
    - Certain classes of problems result in load imbalances even if data is evenly distributed among tasks.
    - When the amount of work each task will perform is intentionally variable, or is unable to be predicted, it may be helpful to use a scheduler – task pool approach. As each task finishes its work, it queues to get a new piece of work.



Designing Parallel Programs

- **Granularity:** a qualitative measure of the ratio of computation to communication. Periods of computation are typically separated from periods of communication by synchronization events.
  - Fine-grain Parallelism:
    - Relatively small amounts of computational work are done between communication events
    - If granularity is too fine it is possible that the overhead required for communications and synchronization between tasks takes longer than the computation
  - Coarse-grain Parallelism:
    - Relatively large amounts of computational work are done between communication/synchronization events
    - In most cases the overhead associated with communications and synchronization is high relative to execution speed so it is advantageous to have coarse granularity.



Designing Parallel Programs
Limits and Costs of Parallel Programming

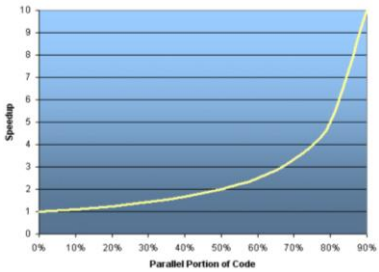
- **Amdahl's Law:** states that potential program speedup is defined by the fraction of code (P) that can be parallelized:

$$speedup = \frac{1}{1 - P}$$

  - If none of the code can be parallelized, P = 0 and the speedup = 1 (no speedup).
  - If all of the code is parallelized, P = 1 and the speedup is infinite (in theory).
  - If 50% of the code can be parallelized, maximum speedup = 2, meaning the code will run twice as fast.
  - Introducing the number of processors performing the parallel fraction of work, the relationship can be modeled by, where P = parallel fraction, N = number of processors and S = serial fraction.

$$speedup = \frac{1}{\frac{P}{N} + S}$$

	speedup		
N	P = .50	P = .90	P = .99
10	1.82	5.26	9.17
100	1.98	9.17	50.25
1000	1.99	9.91	90.99
10000	1.99	9.91	99.02
100000	1.99	9.99	99.90



References

- "Designing and Building Parallel Programs". Ian Foster. http://www-unix.mcs.anl.gov/dbpp/
- "Introduction to Parallel Computing". Ananth Grama, Anshul Gupta, George Karypis, Vipin Kumar. http://www-users.cs.umn.edu/~karypis/parbook/
- "Overview of Recent Supercomputers". A.J. van der Steen, Jack Dongarra. www.phys.uu.nl/~steen/web03/overview.html